2 Data preprocessing

2.1 Introduction

Necessity is the mother of invention

[Plato].

Computational intelligence needs clean, numeric, homogeneous, well organized, and normalized data. This motivates invention of the tools and techniques in the following sections. Various types of data require different preprocessing approaches. The methods presented provide information on the data curation process to prepare data for machine learning.

Data quality is a major concern in big data processing and knowledge management systems. The data preprocessing phase (Pyle, 1999) includes, but is not limited to, converting text, symbols, and characters to numeric values, data imputation, and data cleansing. In this chapter, a general automated approach for data preparation is provided.

2.2 Data preparation

The design and criteria for data cleansing depends on the data. It is very important to select the right curation process. Fig. 2.1 illustrates a general approach to data curation. The presented algorithm consists of the following phases:

2.2.1 Initial cleansing

In this phase we detect and remove any sample (i.e. subject) or variable that has a high ratio of corrupted values. Typically, we set a threshold for missing values and any subject or variable contain erroneous values more than the prespecified threshold we remove them. This method has been applied effectively in (Voth-Gaeddert, Al-Jabery, Olbricht, Wunsch II, Oerther, 2018).

2.2.2 Data imputation and missing values algorithms

Data imputation algorithms has been a popular research topic (Allison, 2012, pp. 1–21; Cahsai, Anagnostopoulos, & Triantafillou, 2015; Efron, 1994; Kantardzic, 2011; King, Honaker, Joseph, & Scheve, 2001; Li, Deogun, Spaulding, & Shuart, 2004, pp. 573–579; Rubin, 1996; Schafer & Schenker, 2000; Stephens & Scheet, 2005; Wasito & Mirkin, 2005;

Computational Learning Approaches to Data Analytics in Biomedical Applications.
https://doi.org/10.1016/B978-0-12-814482-4.00002-4
Copyright © 2020 Elsevier Inc. All rights reserved.

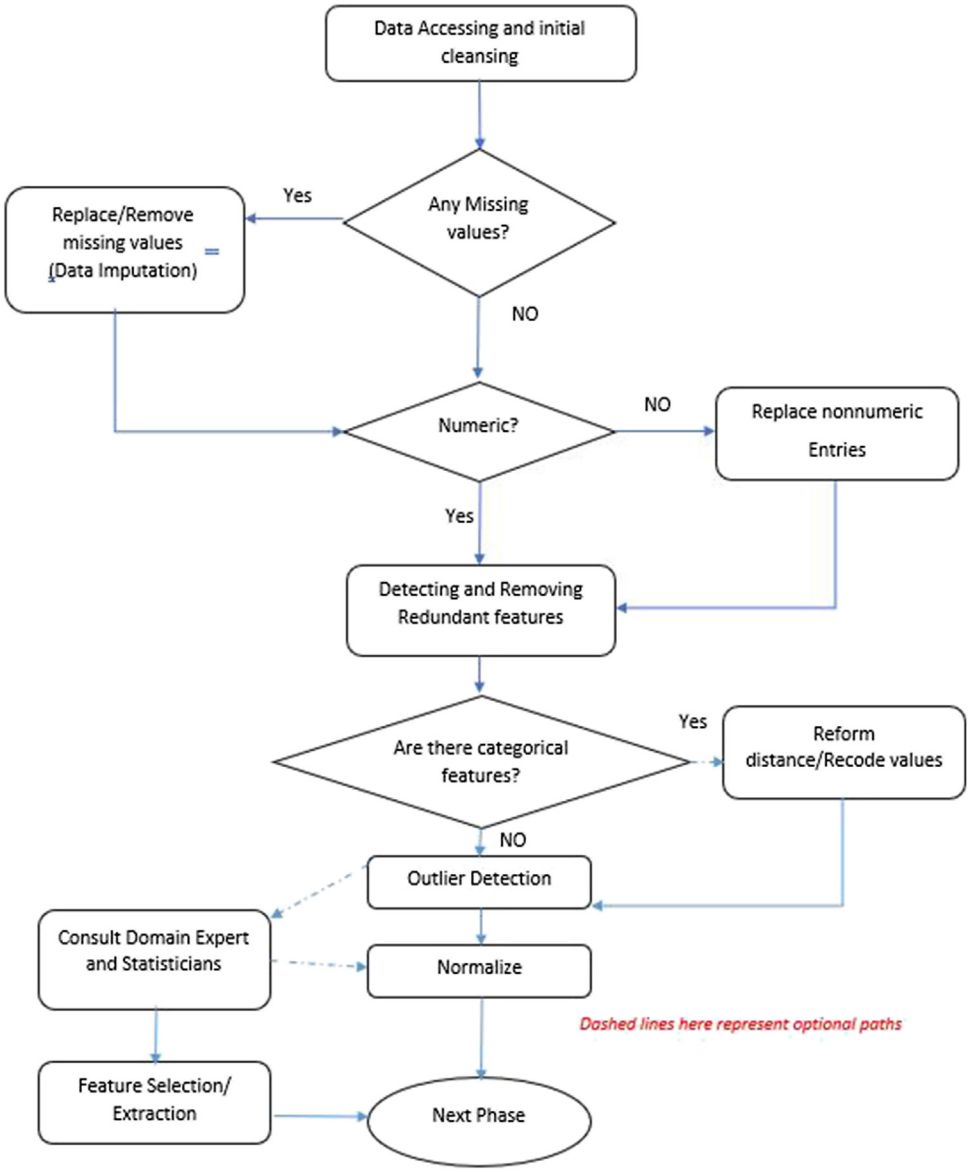


FIG. 2.1 General data preparation paradigm.

Zhou, Zhou, Lui, & Ding, 2014). One relevant problem in data quality is the presence of missing values (MVs). The MV problem should be carefully addressed; otherwise bias might be introduced into the induced knowledge. Common solutions to the MV problem are to either fill-in the MVs (imputation) or ignore/exclude them. Imputation entails a MV substitution algorithm that replaces MVs in a dataset with some plausible values. Imputed data can be used alongside the observed data, but they are only as good as the

assumptions used to create them. Since the imputed data are mostly distilled using the fully observed data, therefore; any discrepancy in the observed data or applying inefficient assumptions (i.e. algorithm) will lead to erroneous data. On the other hand, many computational intelligence and machine learning (ML) techniques (such as neural networks and support vector machines) fail if inputs contain MVs. In almost all real datasets there are missing values. These are due to many reasons: rejecting to answer questions in the case of humanitarian statistics or surveys, sensors malfunctioning, power surges, or bugs in the digitizing process. Combinations of these issues can cause data to be mixed-type, multivalued, or missing (Lam, Wei, & Wunsch, 2015).

Handling missing values is vital in data curation and consequently to the experimental results, since the way missing values are treated will determine the final shape of the dataset. Regardless of the missing values, the final goal is to reach useful and efficient inference about the population of interest. The general methods for handling MVs are discussed in this chapter, along with some advanced methods that are mentioned briefly.

In general, there are three types of MVAs: removal methods where observations (or in some cases variables) with MVs removed; utilization methods which utilize all the available data to cure the dataset from MVs; and imputation methods which attempt to replace or fill in missing data.

2.2.2.1 Removal Methods

These are the simplest methods, since there is no need for complicated or advanced processing. The algorithms simply disregard any observation that has MVs in them. These approaches may lead to biased results, but there are some attempts to avoid that problem. Removing the samples or features that contains many missing values is also an option. However, this could significantly degrade and waste the datasets if not managed properly. MVAs that remove samples that contains MVs are known as **complete-case** or **list-wise deletion** methods. These algorithms are easiest to implement and full data analysis can be applied without modification. Most data processing software packages use these approaches by default. Unfortunately, these methods suffer from bias and loss of precision. This results from the fact that the sample of fully observed cases may not represent the full dataset.

A technique to address this problem is the **Weighted Complete-Case** approach (Zhou et al., 2014). In this method, the sample is reweighted to make it more representative. For example, if the response rate is twice as likely among men as women, data from each man in the sample could receive a weight of 2 in order to make the data more representative. This strategy is mostly used in surveys. This method first creates a model to predict nonresponse in one variable as a function of the rest of the variables in the data. The inverse of the predicted probabilities of response from this model could then be used as survey weights to make the complete-case sample more representative. This method is more efficient in the case of one missing value. It becomes more complicated

for more than one MV and there will be problems with standard errors if predicted response probabilities are close to 0 (Cahsai et al., 2015; Zhou et al., 2014).

Also, one can simply **remove** the samples or features (i.e. variables) that have many missing values in a large dataset. "Many" here means more than a specific threshold. After specifying a threshold of missing values for samples and features, then scan the dataset horizontally and vertically, respectively. So that any sample that has more missing values than the threshold will be removed. The same approach can be followed with features too. Keeping in mind that it is imperative not to sacrifice important features. Thus domain experts must be consulted before removing any feature. However, the code must keep track of all samples and features that have been removed or modified during the cleaning process. It is better to use replacement methods with small datasets and avoid removing any samples from them. This approach drops variables from the analysis that have a large proportion of missing values. This is not recommended since it may exclude important variables in the regression model, required for causal interpretation, and may lead to bias and unnecessarily large standard errors. For important variables with more than trivial amounts of missing data, the explicit model-based methods in the next sections are suggested.

2.2.2.2 Utilization methods

The following methods/algorithms utilize information from both completely and partially observed cases. For example, in repeated measures data, cases with incomplete data at one time point can still provide valid information about the relationship between variables at other time points. Mixed effects models are therefore a popular choice for dealing with longitudinal data with missing outcomes. Because the available case methods make use of more information than the complete-case methods, they are generally better at correcting for bias when data are missing at random than the complete-case methods. The terms "missing at random" and "missing completely at random" are used to describe assumptions about missing data that are needed for standard implementations of multiple imputation but the meanings of these terms are often confused. When observations of a variable are missing completely at random, the missing observations are a random subset of all observations; the missing and observed values will have similar distributions. Missing at random means there might be systematic differences between the missing and observed values, but these can be entirely explained by other observed variables. For example, if blood pressure data are missing at random, conditional on age and sex, then the distributions of missing and observed blood pressures will be similar among people of the same age and sex (e.g. within age/sex strata) (Bhaskaran & Smeeth, 2014). This approach suffers from the fact that different analyses would lead to different subsets of the data, based on the variables used in the analyses process and the pattern of their MVs which will affect the final inference. These methods also suffer from loss of efficiency resulting from disregarding partially observed variables and may be biased especially if the cases with MVs differ systematically from

the fully observed cases. The utilization methods include but are not limited to the following algorithms:

2.2.2.3 Maximum likelihood

The first step in *maximum likelihood* (Allison, 2012, pp. 1−21) estimation is to construct the likelihood function. Suppose that we have n independent case "observations" $(i = 1, \ldots, n)$ on k variables $(y_{i1}, y_{i2}, \ldots, y_{ik})$The fundamental idea behind the maximum likelihood methods is conveyed by their name: find the values of the parameters that are most probable, or most likely, for the data that have actually been observed.

$$L = \prod_{i=1}^{n} f_i(y_{i1}, y_{i2}, \ldots, y_{ik}; \theta)$$

Where $f_i(.)$ is the joint probability (or probability density) function for observation i, and θ is a set of parameters to be estimated. To get the maximum likelihood estimates, we find the values of θ that make L as large as possible. Many methods can accomplish this, any one of which should produce the right result. Now suppose that for a particular observation i, the first two variables, y_1 and y_2, have missing data that satisfy the Missing at Random assumption. (More precisely, the missing data mechanism is assumed to be ignorable). The joint probability for that observation is just the probability of observing the remaining variables, y_{i3} through y_{ik}. If y_1 and y_2 are discrete, this is the joint probability above summed overall possible values of the two variables with missing data:

$$f_i^*(y_{i1}, y_{i2}, \ldots, y_{ik}; \theta) = \sum_{y_1} \sum_{y_2} f_i(y_{i1}, y_{i2}, \ldots, y_{ik}; \theta).$$

For continuous missing variables, integrals will be used instead of summations. Essentially, then, for each observation's contribution to the likelihood function, we sum or integrate over the variables that have missing data, obtaining the marginal probability of observing those variables that have actually been observed. As usual, the overall likelihood is just the product of the likelihoods for all the observations. For example, if there are m observations with complete data and $n - m$ observations with data missing on y_1 and y_2, the likelihood function for the full dataset becomes the following:

$$L = \prod_{i=1}^{m} f_i(y_{i1}, y_{i2}, \ldots, y_{ik}; \theta) \prod_{i=m+1}^{n} f_i(y_{i3}, \ldots, y_{ik}; \theta)$$

where the observations are ordered such that the first m have no missing data and the last n-m have missing data. This likelihood can then be maximized to get maximum likelihood estimates of θ. In the remainder of the paper, we will explore several different ways to do this.

The general algorithm used for finding maximum likelihood estimate of the parameters underlying distribution from a given dataset is the ***Expectation Maximization*** **(*EM*)algorithm** (Dempster, Laird, & Rubin, 1977), summarized in (Zhou et al., 2014)**.**

The **EM algorithm** has two main applications: first, to find the maximum likelihood estimates when data truly have missing values (e.g., patients lost to follow-up); second, to find the maximum likelihood estimates when the likelihood is analytically intractable

and difficult to be optimized, but the likelihood can be simplified by assuming that there exist additional, but missing parameters. The algorithm can be applied whether the missing data are ignorable or not by including a missing data model in the likelihood. The EM algorithm approach is a procedure that iterates between two steps: an expectation step (E step) and a maximization step (M step). The E step calculates the expected value of the full data log-likelihood, given the observed data and current estimated parameters. The M step performs maximization on the expectation of the full data log-likelihood, computed in the E step above. The steps are usually easy to program, and implementation of the EM algorithm is straightforward using standard statistical software packages (Allison, 2012, pp. 1−21).

2.2.3 Imputation methods

Unlike the previous methods which remove incomplete cases or those that have MVs, the imputation algorithms "fill in" MVs. The key realization behind imputation is that, although the MVs are not observed, information about these values may still be extracted from the other observed variables. **Replacement or imputation** (Rubin, 1996) is the most common method and is recommended when working with small datasets. In this method, missing values are replaced with the mean of the feature for continuous features as discussed in Section 2.2.3.1.1. For categorical/discrete features, missing values are replaced with one of the values repeated in the feature. This value is usually chosen randomly, but that it could also be chosen based on the probability distribution of the categories per feature. For example, suppose a dataset has 100 samples (*N=100*), and a feature has the following categories {"a,""b,""c""d," and "e"} with modes {20, 30, 5, 3, and 42}. In this case, does it sound reasonable to give all the categories the same probability to replace a missing value in this feature? The reasonable answer is "No". In this case, a module should be designed to replace the missing values based on a probability function that has been derived using the mode of each category. This approach will maintain the original form of the dataset, minimizing the possible damage that may occur during data compensation.

Algorithm 2.1 describes a generalized approach for cleaning any type of dataset. The Algorithm also detects and removes any mono feature (constant features that take on only one value) since such features are useless in machine learning and keeping them will only waste computation resources and cause more processing delay.

Algorithm 2.1: General data cleansing approach.
Input: Dataset (N,M); *N: number of samples, M: number of features.*
Output: Dataset(N-x_r,M-y_r); x_r, y_r *are number of removed samples features, respectively.*($x_r = y_r = 0$ *initially*).

01: Set H_th = Number of allowed missing values per sample;
02: Set V_th = Number of allowed missing values per feature;
03: Set cat_th: is a threshold help classifies discrete features into categorical or not, typically 10% of N

Phase1: Horizontal checking (optional).

04: For each sample Xi; $i \leq N$
05: If missing values in Xi $> H_th$ *then: remove Xi: increment* x_r;
06: Evaluate Phase1: if $x > 0.1$[a] *N: then increase H_th and repeat Horizontal checking*[b]

Phase2: Vertical checking.

07:For each feature Fj; $j \leq M$
08: If missing values in Fj > V_th then: if (Fj vital) [b] *then: keep; else: Remove Fj repeat 08*
09: If Fj Mono feature: remove Fj and increment y_r*.//removing single value features*
10: find all missing $Mv = \{m_{iv}\}$ *values in Fj*:
11: if Fj categorical/discrete
12: find all categories $C_T = \{c_{k;k=1..C}\}$:
13: for each $c_k \in C_T$: *find* $P(c_k) = (\#\ times\ c_k\ \ appeared)/(N - x_r)$; [c]
14: for each $m_{iv} \in Mv$ *: replace* m_{iv} *with* c_k; $c_k \in C_T$ *based on* $P(c_k)$
15: else: {Fjcontinuous}
16: for each $m_{iv} \in Mv$ *: replace* m_{iv} *with mean (Fj) //after each replacement mean is calculated again*

In general, imputation algorithms can be classified in two types *single* and *multiple* imputation.

2.2.3.1 Single imputation methods

In the following methods, a single value is being imputed for each MV. In general, anytime a single imputation strategy is used, standard errors of parameter estimates will be underestimated since the resulting data are treated as a complete sample, ignoring the consequences of imputation.

2.2.3.1.1 Mean imputation

A popular method replaces each MV with the mean of the correctly recorded values in the same variable. There are two types of this method: the **Unconditional mean** and the **Conditional mean Imputation** methods. The first method replaces MV with the unconditional mean for a specific variable. The second method used for datasets with few missing variables (this method is also known as regression imputation) is an improvement over the unconditional mean imputation. It replaces each MV with the conditional mean of the variable based on the fully observed variables in the dataset.

[a]Categories can be detected in many ways in MATLAB. Readers who are interested in coding and implementation should see Chapter 9 where we explicitly discuss coding tools.
[b]Domain experts should also be consulted at this point too.
[c]Here we can also set equal distribution and.$P(c_k) = \frac{1}{C}; 1 \leq k \leq C$.

For example, if the variable Y is the only variable that has a MV, then a regression model for predicting $\widehat{Y}$ from the other variables can be used to impute Y. The first step fits the model to the cases where it is observed. The second step plugs in the X values for the nonrespondents into the regression equation, obtaining predicted values $\widehat{Y}$ for the missing values of Y. *(Y is the original variable with MVs, $\widehat{Y}$ is the predicted variable values* (Allison, 2012, pp. 1−21; Raghunathan, Lepkowski, Van Hoewyk, & Solenberger, 2001).

Both methods are single imputation approaches, therefore; they have some disadvantages in that they underestimate standard errors and distort the strength of the relationship between variables in the dataset. Conditional mean imputation is acceptable for some problems if the standard errors are corrected (Schafer & Schenker, 2000), but it is problematic if the details of the distribution are of interest (Little & Rubin, 1989). For example, an approach that imputes missing incomes using conditional means tends to underestimate the percentage of cases in poverty.

2.2.3.1.2 Substitution of related observations

In this method, the MV for a specific case is imputed from cases or observations that are related to it. However, this method requires wide knowledge of the relationship between cases in the dataset and applying it may introduce significant errors.

2.2.3.1.3 Random selection

Also known as "Hot Deck imputation". This method was discussed earlier briefly when the case of MVs in categorical variables was described. From experience, this method is practical for imputing MVs in categorical variables rather than for MVs in numerical ones. An advantage of this approach is that it does not require careful modeling to develop the selection criteria for imputing the value, although bias can still be introduced. Note that this approach can serve as a multiple imputation method by selecting multiple values at random from the pool (Cahsai et al., 2015).

2.2.3.1.4 Weighted K-nearest neighbors (KNN) imputation

KNN has many attractive characteristics which made it widely used (Aittokallio, 2009). It does not require the creation of a predictive model for each dimension with MV and takes into account the correlation structure of the data. KNN is based on the assumption that points close in distance are potentially similar. For given input (x_i, w_i) in dataset X with $x_i = \left(z_i, z_i^m\right)$, KNN calculates a weighted Euclidean distance D_{ij} between x_i and $x_j \in X$ such that

$$D_{ij} = \left(\frac{\sum_{k=1}^{d} w_{ik} w_{jk} (x_{ik} - x_{jk})^2}{\sum_{k=1}^{d} w_{ik} w_{jk}} \right)^{1/2}$$

MVs in the kth variable of any observation is estimated by the weighted average of non-MVs of the K most similar observations x_j to x_i, i.e. $\widehat{z}_{ik}^{m} = \sum_{j=1}^{K} \frac{D_{ij}^{-1}}{\sum_{v=1}^{K} D_{iv}^{-1}} x_{jk}$.

2.2.3.2 Multiple imputation

The goal of multiple imputation is to provide statistically valid inference in practical situations when users and data base constructors are distinct in their analyses, models, and capabilities and if the nature of the task prevents accepting any MVs. In multiple imputation as the name implies, multiple values are presented for replacing each MV. These suggested values were drawn from posterior predictive distribution of MVs. The process will lead to having multiple complete datasets. To estimate the parameters of interest for each of the complete datasets, standard full data methods are applied. The resulting estimates are then combined to produce estimates and confidence intervals resulting in multiple estimated parameters and their corresponding standard errors (Rubin, 1996; Zhou et al., 2014).

The multiple imputation method was developed to reduce the standard and estimation errors that may result from the single imputation methods. It was originally developed in the settings of large public-use datasets from sample surveys and censuses. The advances of computational methods and software for creating multiple imputations attracted researchers to the method and participate in publicizing it (Zhou et al., 2014).

2.2.4 Feature enumeration

After curing all the samples in the dataset, the next step is to recode all nonnumeric entries. Usually dataset contains many character-based features; see for example the KDD 99 dataset in the (Tavallaee, Bagheri, Lu, & Ghorbani, 2009). In such cases, the proper method of processing such features needs to be determined. In general, a dataset X that has n samples and m features that are mixed categorical and numerical features can be represented as follows:

$X = [fc_1, fc_2, \ldots fc_r, fn_1, fn_2, \ldots, fn_{m-r}]$; Where r is the number of categorical features in X and $(m-r)$ is the number of numerical features.

In this section, the effect of different enumeration techniques on the performance of support vector machine (SVM) based anomaly detector will be demonstrated. There multiple ways for enumerating categorical features. Let us assume a categorical feature fc_u $(u = 1 \ldots r)$, that has a domain of k values $\{c_1, c_2, \ldots, c_k\}$ (See Algorithm 2.1 above). The following methods show all the possible enumeration techniques for such feature.

- ASCII conversion: This method works perfectly on symbolic features as discussed in Table 2.1. Where all symbols in the feature are replaced with their ASCII code. This can be simply summarized as follows:

  For each cell i in fc_u

  $fc_u^i = ASCII(fc_u^i)$ For symbolic feature or.

  $fc_u^i = \sum_{j=1}^{s} fc_u^i(j)$ *for string features* ; where j is the index of characters in the string.

  Another approach can be achieved by mapping string values to symbolic and then replace them with their ASCII values.

Table 2.1 Results and comparison among different SVM models.

Data Normalized?	Yes															
Symbolic mapping	Numerical Mapping								ASCII							
SVM Kernel Fn.	Sigmoidal				RBF				Sigmoidal				RBF			
SVM Bx. Constraints	Inf	3500	1500	500	Inf	3500	1500	500	Inf	3500	1500	500	Inf	3500	1500	500
Accuracy %	69	69.69	69.69	69.69		43	43	43	69	69.69	61.76	69.7	Algorithm didn't converge!	43	42.98	43
Precision%	93.89	91.68	91.68	91.68		0.08	Na	13	93.89	91.68	97.86	91.7		0.087	0.04	0.09
Recall%	48.77	51.4	51.41	51.4		0	0	$2x10^{-4}$	48.77	51.41	33.55	51.4		0	$8x10^{-5}$	$2x10^{-4}$
F1-Score%	64.2	65.88	65.88	65.88		0	Na	$4x10^{-4}$	64.2	65.88	49.97	65.9		0	$2x10^{-4}$	$3x10^{-4}$
Iters to Converge	24367	44969	44968	44965		15612	12441	7427	24367	44969	55142	44965		13541	12603	13541
Training Time (s)	1261.3	2548	2546	2855		838	760	605	2232	--	--	1560		484	493	471
No. SVs	29858	30431	30431	30431		ig	ig	4087	29858	24468	24468	30431		3950	4199	4262
Performance evaluation on 20% of the training data ↓																
Accuracy %	76.36	75.78	75.78	75.78		ig	ig	99.99	22.59	80.75	80.7	23		53.39	53.39	53.39
Precision %	74.68	73.11	73.11	73.11		ig	ig	99.97	29.1	78.49	78.49	29.6		Na	Na	Na
Recall %	74.58	75.98	75.98	75.98		ig	ig	100	46	80.85	80.85	47.2		0	0	0
F1-Score %	74.63	74.52	74.52	74.52		ig	ig	99.99	35.66	79.65	79.65	36.4		Na	Na	Na

(*continued on next page*)

- Numerical encoding: This method applies to all categorical and multi-valued discrete features. In this method, symbolic and string values are mapped into numeric indices. The values of the indices can simply be specified in sequence as 0,1, …etc. Based on the appearance of the symbol in the feature (i.e. first symbol found will mapped to 0, second to 1, and so on).

  For each cell i infc_u
  If $fc_u^i == c_{ind}$then $fc_u^i = ind$; $1 \leq ind \leq k$

The above method is simple and easy to implement. But it does not provide efficient encoding for all categorical features. Features with categories that have different distances from each other cannot be encoded using this method. Therefore; the method was modified for such features. This was achieved by careful selection of numerical values based on the meaning of distance between categories. For example, race and ethnicity, which are very common in biomedical datasets, are symbolic categorical features.

Table 2.1 *—cont'd*

Data Normalized?	No															
Symbolic mapping	Numerical Mapping								ASCII							
SVM Kernel Fn.	Sigmoidal				RBF				Sigmoidal				RBF			
SVM Bx. Constraints	Inf	3500	1500	500	Inf	3500	1500	500	Inf	3500	1500	500	Inf	3500	1500	500
Accuracy %	71	70.5	71.6	70.7	43	70	73.5	73.1	73	73.3	72.9	68	43	71.97	72.2	72.18
Precision%	89	96.1	88.55	92.9	Na	97.59	79.5	97.5	85.2	91.25	91.3	96.27	Na	97.24	97.5	97.5
Recall%	55.98	50.2	57.55	52.57	0	48.5	54.8	54	63.9	58.7	57.9	45.6	0	52.24	52.4	52.46
F1-Score%	68.77	65.96	69.77	67.15	Na	64.81	70	69.61	73	71.45	70.9	61.88	Na	67.97	68.2	68.22
Iters to Converge	25006	19995	46181	28114	242	5229	14721	7637	23143	26227	24539	18186	702	21318	15883	10602
Training Time (s)	2408	1189	2821	1595	ig	279.5	847	585	2091	1640	1485	706	ig	884.6	797.6	717.7
No. of SVs	30057	11909	30615	18266	167	1870	3710	3627	24833	16714	15669	11522	306	3271	3503	3676
Performance evaluation on first 20% of the training data ↓																
Accuracy %	76.19	70.6	75.47	72	53.4	99.2	99.96	99.96	80.4	80	80	64.5	53.4	99.94	99.94	99.93
Precision%	74.51	81.67	72.92	77.5	88.88	99.95	99.93	99.94	79	82.7	83.39	76.9	100	99.91	99.9	99.9
Recall%	74.34	47.6	75.35	56.6	0	98.23	99.97	99.97	78.9	72.13	71.29	34	0	99.97	99.97	99.94
F1-Score%	74.4	60.15	74.12	65.5	0	99	99.95	99.96	78.98	77.09	76.87	47.2	0.001	99.94	99.93	99.92

Table 2.1 showed the effect of using different data processing techniques on the classification performance of the SVM and RBF. Where the best classifier was found using numerical mapping for the symbolic features and RBF as a kernel function for the SVM classifier.

In this case, a vector of weights $W = \{w_1, w_2, \ldots w_k\}$ will be constructed such that the values of the weights are selected based on the distances between the categories in the feature. These distances are specified either statistically (i.e. based on the statistic characteristics of the variables) or scientifically using a paradigm derived from the meanings of each variable and how far are they from each other.

For each cell i in fc_u

If $fc_u^i == c_{ind}$ then $fc_u^i = w_{ind}$; $1 \leq ind \leq k$

This can be considered as a generalized case of binary encoding presented in (Lam et al., 2015), which is described further below.

- Binary feature transformation is presented to handle multivalue categorical features by setting the corresponding entries in the binary vector. Consider a categorical feature fc_u that has a domain of $l$values $\{d_1, d_2, \ldots d_l\}$. In the binary vector $[b_1, b_2, \ldots b_l]$, Each b_v corresponds to each domain value d_v. This approach is implemented by providing a frequency vector $f^i = \left[f_1^i, f_2^i, \ldots f_l^i\right]$ in each cluster C_i for each categorical feature fc_u, where

$$f_v^i = \frac{\sum_{X_k \in C_i} b_v^k}{\sum_{v=1}^{l} \sum_{X_k \in C_i} b_v^k} \quad v = 1 \ldots l$$

Where: b_v^k is binary representation of d_v for sample X_k that belongs to cluster C_i. Clearly, $0 \leq f_v^i \leq 1$ and $\sum_{v=1}^{l} f_v^i = 1$.

Furthermore, missing values can be resolved by setting all of the binary entries to 1. One form of uncertainty in this feature occurs when data are specified by a range of values, instead of one scalar. To correct this problem, if a numerical feature fn_u has interval data [a, b], it is represented by two numeric features, $fn_u^1 = a$and $fn_u^2 = b$. Missing values of numeric features are replaced by the average of the observed value or by the k-nearest neighbors (Lam et al., 2015; Obafemi-Ajayi, Lam, Takahashi, Kanne, & Wunsch, 2015). This method can be considered as special case of the well-known *one-of-n remapping* method. (Pyle, 1999)

2.2.4.1 *Special cases of categorical data representation using COBRIT traumatic brain injury data as an example*

In medical data, certain lab measurements or drug intake are represented (or encoded) using histogram bins. This could imply that there is not a significant difference in a small change between values within a given range. For example, consider the data available from the citicoline brain injury treatment trial (COBRIT) on the effects of citicoline on traumatic brain injury (TBI) (Zafonte et al., 2012). This data set is publicly available by permission from Federal Interagency Traumatic Brain Injury Research (FITBIR) Informatics System website (NIH, 2019). The features representing drug intake are quantified in histogram bins (such as: no dose, < 25 cc, ≥ 22 cc). This type of data is ordinal in nature, not categorical, as distances between the bins do matter. It would be more appropriate to represent the features using integer values.

Another scenario could be categorical data that has multiple combinations of possible values. For example, in the COBRIT data, location information of injury on the brain from the CT scan. Some of the phenotypes denoting this had over 70 different possible states or categories. Encoding that many variations are potentially impactful on

a machine learning model's performance. The proposed approach encodes the information as a bit-type representation. To illustrate this concept, we utilize the computed tomography (CT) intraparenchymal lesion anatomic site feature from the COBRIT data. An injury could be located in one or more of the following locations: frontal (left/right), temporal (left/right), occipital (left/right), parietal (left/right), brainstem, and cerebellar. To determine the total number of bits most efficient for the encoding of the feature, we consider each individual possible site. For brainstem and cerebellar, given that the injury was either at that position or not respectively, a single bit representation is applied. If the lesion is present, the bit is on (1) otherwise off (0). For the locations (frontal, temporal, occipital, and parietal) that also had a right or left component, a 2-bit representation is needed. The lesions could exist either on the left or right side, or both simultaneously. For example, if a patient had the location on the right frontal, this would be encoded as 01. If the patient had the location on the left frontal, it is encoded as 10. A bit representation of 11 implies both left and right. Thus, given 6 possible locations, with four having both left and right components, a total of 10 bits is needed to efficiently and robustly represent this categorical feature with over 80 distinct values. This approach provides a meaningful but concise representation for complex categorical data. For example, an entry of ['Left frontal;Right frontal;Left temporal;Right temporal;Right parietal;Brainstem/diencephalon/CC'] will be encoded as 0,011,110,101. The first 2 bits imply nothing in the occipital region, next 2 bits denote the right and left frontal regions, next 2 bits denote the right and left temporal regions, while next 2 bits implies presence in only right parietal region. The last 2 bits indicate whether present or not in the cerebellar and brainstem regions respectively.

2.2.5 Detecting and removing redundant features

This process includes, but is not limited to removing highly correlated features. Redundant features are those who show a specific degree of correlation with each other. **Correlation** in general, is an attempt to characterize the strength and the direction of an association between two variables. In terms of the strength of relationship, the value of the correlation coefficient varies between 0 and 1 in absolute value. When the absolute value of the correlation coefficient lies around 1, then it is said to be a perfect degree of association between the two variables and hence one of these features is not required and should be removed. The direction of the relationship is simply the + (indicating a positive relationship between the variables) or - (indicating a negative relationship between the variables) sign of the correlation or it can be shown as null. In summary, a positive association between variables means they are proportional with each other (*an increase in one lead to an increase in the other*). However, a correlation near zero does not necessarily mean a weak association—in the case of Pearson correlation it would just mean a weaker linear association (but a nonlinear relationship may exist). As an example of this case, the *severity measure* variable has positive correlation with Autism Diagnostic Observation Schedule (ADOS) variable in an Autism dataset (Al-Jabery et al., 2016). While

a negative association means each variable is proportional to the reciprocal of the other, an increase in one leads to a decrease in the other and vice versa (*e.g.* the ADOS communication with the IQ in autism) (Al-Jabery et al., 2016). However, it is very important to note that correlation does not equal causality. In other words, a null correlation doesn't mean the variables are independent and a strong association does not prove that a change in one variable causes a change in the other variable. In fact, statistics such as the correlation or other advanced statistics mainly provide us with clues regarding what the plausible causal relationships might be, as discussed in many statistics literature (e.g. (Chen & Popovich, 2002; Glass & Hopkins, 1996; Pedhazur & Schmelkin, 1991)). There are several types of correlation indices in statistics such as: Pearson correlation, Kendall rank correlation, Spearman correlation, and the Point-Biserial correlation (Cohen, Cohen, West, & Aiken, 2013; Correlation (Pearson, Kendall, Spearman), 2017). The Pearson and Spearman correlation are discussed below, since they are commonly used metrics for determining correlation. The reader can see (Chen & Popovich, 2002) or (Glass & Hopkins, 1996) for explicit details and discussion on additional correlation indices.

2.2.5.1 Pearson correlation

Is also known as the "product−moment correlation" because it was calculated by multiplying the z scores of two variables and then calculating the average of these products based on a group of n cases. Pearson "***r***" correlation is the most widely used correlation statistic to measure the degree of the linear relationship between variables. It has been estimated that Pearson's *r* and its special cases are chosen 95% of the time in research to describe a relationship or to infer a population correlation (Glass & Hopkins, 1996). Pearson's *r* correlation is calculated using the formula:

$$r = \frac{\sum X_i Y_i - \frac{\sum X_i \sum Y_i}{n}}{\sqrt{\left(\sum X_i^2 - \frac{(\sum X_i)^2}{n}\right)\left(\sum Y_i^2 - \frac{(\sum Y_i)^2}{n}\right)}}$$

where:

n: is the number of values in each dataset,
X_i& Y_i: are the paired variables.

Pearson's *r* is viewed as an indicator that describes a linear interdependence between variables X and Y, with the form of $Y = a + (b) \times X$, where a and b are constants. As illustrated earlier, the closer the absolute value of "r" is to 1, the more linear interdependence between the variables. Values near 0 indicate a lack of linear relationship between the variables, but does not rule out the possibility of a nonlinear relationship.

2.2.5.2 Spearman correlation

Spearman correlation is known as Spearman's "rank correlation" coefficient since it is found by calculating the Pearson correlation on the ranked data within two features.

Spearman's correlation is often denoted by the Greek letter ρ and it provides a measure of the strength and direction of a monotonic associations between ranks of the two features. It is sometimes used as an alternative to the Pearson correlation coefficient since it is less sensitive to outliers due to the utilization of ranks rather than actual data values in the calculation and can capture the strength of monotonic relationships beyond linear associations (Conover, 1999).

A **threshold** between 0 and 1 will be chosen such that if the absolute value of the correlation between a pair of features exceeds the threshold, one of the features in the pair will be removed. However, the threshold of correlation depends on many factors, including domain expert opinion, quality of the final results, etc. In Chapter 9, the code implementation for automatic removal of the correlated features is presented.

Mono features or single value features should be removed from the dataset at this point to avoid unnecessary processing of them. These features have only one value or only few samples for which they vary (i.e. <1% or so). Such features will not have any advantage during the clustering process and should be removed.

2.2.6 Recoding categorical features

This step is vital in data analysis, since it is almost impossible for traditional machine learning tools to operate on a nonnumerical dataset. Therefore, features which are binary or nominal must be recoded into a numerical (i.e. floating numbers) form. The binary encoding method described above is a perfect tool for this job. However, there are some algorithms that deal with categorical features clustering such as the k-dimensional clustering algorithm (Al-Jabery et al., 2016). This algorithm can handle categorical features without the need to recode or normalize them.

2.2.7 Outlier detection

Outliers are samples which deviate extremely from other data samples. The process of detecting outliers is also known as *anomaly detection*. Outlier detection is important in medical applications as well as many other applications such as credit card fraud detection, intrusion detection, fault detection in smart grids, image processing, network surveillance, and any application that requires concentrating on uncommon activities/phenomena. Outlier detection can be considered as the complement of the clustering process, since the former looks for the anomalies and minorities while the latter focuses on the larger groups (data majority) and clusters them in groups based on how similar they are to each other. *Clustering will be discussed in* Chapter 3.

It is important to distinguish between outliers and noisy data. Outliers are genuine objects in the data set that have unique characteristics that isolate them from the remaining objects. While noisy data result from erroneous data that randomly appear in datasets. Outliers reveal that they have been generated by a mechanism different than the one that generated the remaining data. Fig. 2.2 illustrates how outliers look like in a neuroimaging dataset (Obafemi-Ajayi et al., 2017).

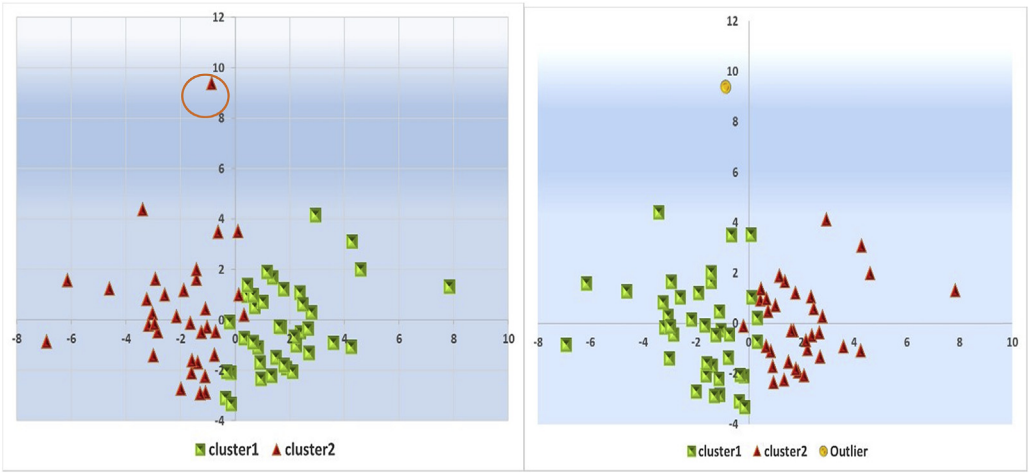


FIG. 2.2 Shows how outliers can be detected through visualizing the dataset.

There are many outlier detection techniques, some are statistical, proximity based, and clustering based methods (Han, Kamber, & Pei, 2012). In fact, for small datasets, outliers can be detected visually as shown in Fig. 2.2, where the data has been visualized using principal component analysis as discussed in Chapter 8. In the figure on the left the outlier classified as part of cluster 2 is far away from any other samples. After using a different clustering approach, it has been classified alone which is the most common case in outlier detection.

2.2.8 Normalization

Normalizing data is essential in accelerating the machine learning process and improving results. Most ML tools require the data to be normalized for example neurons in most neural-networks—based tools require the range of the input to be in the range of [0,1] or [-1, +1], depending on the type of neuron (Haykin, 2013). Even tools that do not require data normalization may benefit from it. For instance Pyle (Pyle, 1999) has shown that exposing information and easing the learning task can reduce the *feature swamping* effect (Nedumov, Turdakov, Maiorov, & Ovchinnikov, 2013; Sled, Zijdenbos, & Evans, 1998; Sola & Sevilla, 1997; Tan & Triggs, 2010).

In general, data normalization is the process of remapping the data variables from one range to another unified range for all variables. It also deals with various issues such as variable distribution, since some methods also adjust and correct for range and distribution. However, there are some challenges that affect the normalization process such as out of range values. It is recommended to remove such samples from the set if possible. If one wants data in the range [0,1], the following formula can be used:

$$x_{normalized} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

This method is also called normalization. The other approach is called standardization and it used to generate data variables with zero mean and unit variance. To achieve this, the following formula is used:

$$x_{new} = \frac{x - \mu}{\sigma}$$

Where

μ: The statistical mean
σ: The standard deviation.

As mentioned above, outliers can significantly affect both methods. Where in normalization it will squeeze the normal data into the lowest portion of the range (i.e. very close to zero) while in standardization it will make the data unbounded (Kantardzic, 2011). Therefore; It is highly recommended to remove outliers "*if applicable*" before normalizing the data. Sections 9.3.3 in Chapter 9 illustrates the implementation of outlier removal.

2.2.9 Domain experts

This phase is manual and recommended when dealing with data types for the first time. It is highly recommended that domain experts to be consulted whenever there is a significant modification to the data, such as when removing or selecting features or considerable amount of subjects.

2.2.10 Feature selection and extraction

The nature of the selected features before applying clustering algorithms play vital role in clustering and detecting patterns since it concentrates the clustering process on specific features in the data. Therefore; these features should reflect the goal of the clustering process or the project in general. For example, if it is of interest to cluster a group of objects based on their mass density then weights and volumes should be extracted, while transparency is of interest then color and surface area should be the selected features. *Feature selection* is choosing a subset of existing features from the data for subsequent use while maintaining their original/physical meaning. While *feature extraction* may include generating new features that provide better numerical representation to the patterns in the data and its structure. Yet these extracted features may not have any physical meaning. However, in the literature these two terms are often used interchangeably without distinguishing between them. Both approaches are of great importance in data clustering. As pointed out by (Jain, Murty, & Flynn, 1999) feature selection is an ad hoc process and often involves a trial and error process to test various subsets. Therefore; we have developed in (Al-Jabery et al., 2016) an algorithm that selects the feature subset automatically and determines the most optimal set based on specific evaluation metrics. Some feature extraction methods, such as principal component

analysis, do not require evaluation metrics and can be used directly. This technique will be discussed in Chapter 8. Benefits of feature selection and extraction can be summarized by the following:

- Decrease the storage requirements and measurement cost
- Simplify the subsequent design process
- Facilitate the understanding of the data
- Dimensionality reduction helps in visualizing the data.

Feature selection can also be executed based on domain experts′ selection of the most important subset of the data (Xu & Wunsch, 2009).

After implementing all the previous phases the data are ready for advanced processing and analysis.

2.3 Example

An example of the effect of using different data curation/processing techniques is shown in Table 2.1.

The data set used for training the ANNs is the modified version of KDD′99 dataset, or what is called NSL-KDD. NSL-KDD is a data set suggested to solve some of the inherent problems of the KDD′99 data set which are mentioned in (Tavallaee et al., 2009). This new version of the KDD data set still suffers from some of the problems discussed in (McHugh, 2000) and may not be a perfect representative of existing real networks. However, because of the lack of public datasets for network-based IDSs, it still can be applied as an effective benchmark data set to help researchers compare different intrusion detection methods. This data set consists of **125,973** samples for training and **22,544** samples used for testing. Each sample has 41 features. These features consist of 3 symbolic features and the rest are all numeric. However, the KDD-NSL dataset has the following advantages over the original KDD dataset:

- It does not include redundant records in the training set, so the classifiers will not be biased toward more frequent records.
- There are no duplicate records in the proposed test sets; therefore, the performance of the learners are not biased by the methods which have better detection rates on the frequent records.
- The number of selected records from each difficulty-level group is inversely proportional to the percentage of records in the original KDD data set. As a result, the classification rates of distinct machine learning methods vary in a wider range, which makes it more efficient to have an accurate evaluation of different learning techniques.
- The number of records in the training and test sets are reasonable, which makes it affordable to run the experiments on the complete set without the need to

randomly select a small portion. Consequently, evaluation results of different research works will be consistent and comparable.

In this example, 32 different tests have been performed to select the best SVM architecture for anomaly detection. The experimental results are shown in Table 2.1 below. The results were divided into two main parts based on the normalization of the data: results from normalized data and those from using the real values of the data. Furthermore, these two parts are also partitioned into two sections based on the mapping used to map the symbolic features into ASCII and numerical mapping. Another division is performed based on the kernel function used and finally the last level of partitioning is based on the number of Box constraints that used to train the SVM which has a great effect on the classification performance. The entries in Table 2.1 represent: statistical measurements for the classifier, time required for training, number of support vectors generated. The table is divided horizontally also into two sections: the upper half contains the performance of the classifier on the training set while the lower one shows its performance on the training set. Please, note that some entries are marked as "ig" this is stands for "ignored" because of the poor performance of the corresponding classifier model.

2.4 Summary

This chapter has covered the process of data curation through its different phases. A general data curation/cleansing framework was presented which can fit a broad variety of datasets. Raw data prior to cleansing and curation is usually not ready for distilling correct inferences. This chapter's framework describes techniques for data curation, imputation, feature extraction, correlation analysis, and practical application of these algorithms. We also provided techniques that have been developed from our experience in data processing. Finally, we presented a practical example showing the effect of using different imputation methods on the performance and efficiency of SVM.

References

Aittokallio, T. (2009). Dealing with missing values in large-scale studies: Microarray data imputation and beyond. *Briefings in Bioinformatics, 11*(2), 253–264. https://doi.org/10.1093/bib/bbp059.

Al-Jabery, K., Obafemi-Ajayi, T., Olbricht, G. R., Takahashi, T. N., Kanne, S., & Wunsch, D. (2016). Ensemble statistical and subspace clustering model for analysis of autism spectrum disorder phenotypes. In *International conference of the IEEE engineering in medicine and biology society (EMBC)* (pp. 3329–3333). https://doi.org/10.1109/EMBC.2016.7591440.

Allison, P. D. (2012). *Handling missing data by maximum likelihood. SAS globalforum 2012 statistics and data analysis*. https://doi.org/10.1042/CS20080236.

Bhaskaran, K., & Smeeth, L. (2014). What is the difference between missing completely at random and missing at random? *International Journal of Epidemiology, 43*(4), 1336–1339. https://doi.org/10.1093/ije/dyu080.

Cahsai, A., Anagnostopoulos, C., & Triantafillou, P. (2015). Scalable data quality for big data: The pythiaframework for handling missing values. *Big Data, 3*(3), 159−172. https://doi.org/10.1089/big.2015.0002.

Chen, P., & Popovich, P. (2002). Correlation: Parametric and nonparametric measures. In *Sage university papers series on quantitative applications in the social sciences. 2455 Teller Road, Thousand Oaks California 91320 United States of America*. SAGE Publications, Inc. https://doi.org/10.4135/9781412983808.

Cohen, J., Cohen, P., West, S. G., & Aiken, L. S. (2013). *Applied multiple regression/correlation analysis for the behavioral sciences* (3rded.). New York, NY, US: Routledge https://doi.org/10.4324/9780203774441.

Conover, W. J. (1999). *Practical nonparametric statistics* (3rded.). New York, NY: Wiley.

Correlation (Pearson, Kendall, Spearman).(2017). Retrieved June 1, 2019, fromhttps://www.statisticssolutions.com/correlation-pearson-kendall-spearman/.

Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EMalgorithm. *Journal of the Royal Statistical Society: Series B (Methodological), 39*(1), 1−22. https://doi.org/10.1111/j.2517-6161.1977.tb01600.x.

Efron, B. (1994). Missing data, imputation, and the bootstrap. *Journal of the American Statistical Association, 89*(426), 463−475. https://doi.org/10.1080/01621459.1994.10476768.

Glass, G. V., & Hopkins, K. D. (1996). *Statistical methods in education and psychology* (3rded.). Needham Heights, MA, US: Allyn& Bacon.

Han, J., Kamber, M., & Pei, J. (2012). *Data mining: Concepts and techniques* (3rded.). Elsevier https://doi.org/10.1016/C2009-0-61819-5.

Haykin, S. (2008). Neural Networks and Learning Machines. *Pearson Prentice Hall New Jersey USA 936 pLinks, 3*. https://doi.org/978-0131471399.

Jain, A. K., Murty, M. N., & Flynn, P. J. (1999). Data clustering: A review. *ACM Computing Surveys, 31*(3), 264−323. https://doi.org/10.1145/331499.331504.

Kantardzic, M. (2011). *Data mining: Concepts, models, methods, and algorithms* (2nded). Hoboken, New Jersey, US: Wiley.

King, G., Honaker, J., Joseph, A., & Scheve, K. (2001). Analyzing incomplete political science data: An alternative algorithm for multiple imputation. *American Political Science Review, 85*(1269), 49−69. https://doi.org/10.2307/3117628.

Lam, D., Wei, M., & Wunsch, D. (2015). Clustering data of mixed categorical and numerical type with unsupervised feature learning. *IEEE Access, 3*, 1605−1616. https://doi.org/10.1109/ACCESS.2015.2477216.

Li, D., Deogun, J., Spaulding, W., & Shuart, B. (2004). *Towards missing data Imputation : A study of fuzzy K-means clustering method*. Springer-Verlag Berlin Heidelberg,(c). https://doi.org/10.1007/978-3-540-25929-9_70.

Little, R. J. A., & Rubin, D. B. (1989). The analysis of social science data with missing values. *Sociological Methods & Research, 18*(2−3), 292−326. https://doi.org/10.1177/0049124189018002004.

McHugh, J. (2000). Testing intrusion detection systems: A critique of the 1998 and 1999 DARPA intrusion detection system evaluations as performed by lincoln laboratory. *ACM Transactions on Information and System Security, 3*(4), 262−294. https://doi.org/10.1145/382912.382923.

Nedumov, Y. R., Turdakov, D. Y., Maiorov, V. D., & Ovchinnikov, P. E. (2013). Automation of data normalization for implementing master data management systems. *Programming and Computer Software, 39*(3), 115−123. https://doi.org/10.1134/S0361768813030055.

NIH. (2019). *Federal interagency traumatic brain injury research (FITBIR) Informatics System*. Retrieved June 1, 2019, fromhttps://fitbir.nih.gov/.

Obafemi-Ajayi, T., Al-Jabery, K., Salminen, L., Laidlaw, D., Cabeen, R., Wunsch, D., et al. (2017). Neuroimaging biomarkers of cognitive decline in healthy older adults via unified learning. In *2017 IEEE symposium series on computational intelligence (SSCI)* (pp. 1–9). https://doi.org/10.1109/SSCI.2017.8280937.

Obafemi-Ajayi, T., Lam, D., Takahashi, T. N., Kanne, S., & Wunsch, D. (2015). Sorting the phenotypic heterogeneity of autism spectrum disorders: A hierarchical clustering model. In *IEEE conference on computational intelligence in bioinformatics and computational biology (CIBCB)* (pp. 1–7). IEEE. https://doi.org/10.1109/CIBCB.2015.7300337.

Pedhazur, E. J., & Schmelkin, L. P. (1991). *Measurement, design, and analysis: An integrated approach* (1sted.). New York: Taylor & Francis.

Pyle, D. (1999). *Data preparation for data mining* (1sted.). Morgan Kaufmann.

Raghunathan, T. E., Lepkowski, J. M., Van Hoewyk, J., & Solenberger, P. (2001). A multivariate technique for multiply imputing missing values using a sequence of regression models. *Survey Methodology, 27*, 85–95.

Rubin, D. B. (1996). Multiple imputation after 18+ years. *Journal of the American Statistical Association, 91*(434), 473–489. https://doi.org/10.1080/01621459.1996.10476908.

Schafer, J. L., & Schenker, N. (2000). Inference with imputed conditional means. *Journal of the American Statistical Association, 95*(449), 144–154. https://doi.org/10.1080/01621459.2000.10473910.

Sled, J. G., Zijdenbos, A. P., & Evans, A. C. (1998). A nonparametric method for automatic correction of intensity nonuniformity in MRI data. *IEEE Transactions on Medical Imaging, 17*(1), 87–97. https://doi.org/10.1109/42.668698.

Sola, J., & Sevilla, J. (1997). Importance of input data normalization for the application of neural networks to complex industrial problems. *IEEE Transactions on Nuclear Science, 44*(3), 1464–1468. https://doi.org/10.1109/23.589532.

Stephens, M., & Scheet, P. (2005). Accounting for decay of linkage disequilibrium in haplotype inference and missing-data imputation. *The American Journal of Human Genetics, 76*(3), 449–462. https://doi.org/10.1086/428594.

Tan, X., & Triggs, B. (2010). Enhanced local texture feature sets for face recognition under difficult lighting conditions. *IEEE Transactions on Image Processing, 19*(6), 1635–1650. https://doi.org/10.1109/TIP.2010.2042645.

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. In *IEEE symposium on computational intelligence for security and defenseapplications, CISDA 2009*. https://doi.org/10.1109/CISDA.2009.5356528.

Voth-Gaeddert, L. E., Al-Jabery, K. K., Olbricht, G. R., Wunsch, D. C., II, & Oerther, D. B. (2018). Complex environmental associations with child diarrhea and stunting. *Environmental Science*.

Wasito, I., & Mirkin, B. (2005). Nearest neighbour approach in the least-squares data imputation algorithms. *Information Sciences*. https://doi.org/10.1016/j.ins.2004.02.014.

Xu, R., & Wunsch, D. C. (2009). *Clustering.Computational intelligence*. https://doi.org/10.1002/9780470382776.

Zafonte, R. D., Bagiella, E., Ansel, B. M., Novack, T. A., Friedewald, W. T., Hesdorffer, D. C., et al. (2012). Effect of citicoline on functional and cognitive status among patients with traumatic brain injury. *JAMA, 308*(19), 1993–2000. https://doi.org/10.1001/jama.2012.13256.

Zhou, X.-H., Zhou, C., Lui, D., & Ding, X. (2014). *Applied missing data analysis in the health sciences* (1sted.). Wiley.